

DESAFIO PRÁTICO

RxJS: Sistema de Monitoramento Reativo

Construa um pipeline reativo completo com Observables, operadores e tratamento de erros

Nível	Tempo Estimado	Pontuação	Entrega
Intermediário	4 a 8 horas	100 pontos	GitHub + README

1. Contexto do Desafio

Você foi contratado por uma empresa de logística para desenvolver um sistema de monitoramento de entregas em tempo real. O sistema precisa processar múltiplos fluxos de eventos simultaneamente: atualizações de GPS dos entregadores, notificações de status dos pedidos e alertas de atraso.

Seu trabalho é construir o núcleo reativo desse sistema utilizando RxJS, aplicando os conceitos estudados em aula de forma integrada.

Dica: não existe uma única solução correta. O que será avaliado é a qualidade do pipeline reativo, não apenas se o código funciona.

2. Setup do Projeto

2.1 Estrutura esperada

Crie um projeto Node.js com a seguinte estrutura de arquivos:

```
rxjs-desafio/  
├── src/  
│   ├── streams/  
│   │   ├── gps.stream.ts  
│   │   ├── pedidos.stream.ts  
│   │   └── alertas.stream.ts  
│   ├── operadores/  
│   │   └── custom.operadores.ts  
│   ├── utils/  
│   │   └── simulador.ts  
│   └── main.ts  
├── package.json  
└── README.md
```

2.2 Instalação

Execute os comandos abaixo para iniciar o projeto:

```
npm init -y
npm install rxjs
npm install -D typescript ts-node @types/node
npx tsc --init
```

Você pode usar JavaScript puro se preferir, mas TypeScript é incentivado.

3. Tarefas

Tarefa 1 — Criando os Streams de Dados (20 pts)

Crie um simulador que gere dados falsos para os três streams abaixo. Use os operadores de criação do RxJS (não invente dados na mão dentro do subscribe).

Stream 1: GPS dos Entregadores

Crie um Observable chamado `gps$` que emita a cada 1 segundo um objeto com a seguinte estrutura:

```
// Exemplo de emissão esperada:
{
  entregadorId: string,    // ex: 'ENT-001', 'ENT-002'
  lat: number,             // latitude simulada
  lng: number,             // longitude simulada
  velocidade: number,      // km/h entre 0 e 80
  timestamp: Date
}
```

Stream 2: Status dos Pedidos

Crie um Observable chamado `pedidos$` que emita a cada 2 segundos um objeto com:

```
{
  pedidoId: string,
  status: 'coletado' | 'em_rota' | 'entregue' | 'falhou',
  entregadorId: string,
  timestamp: Date
}
```

Stream 3: Alertas do Sistema

Crie um Observable chamado `alertas$` que emita eventualmente (intervalo aleatório entre 3s e 8s) com:

```
{
  tipo: 'atraso' | 'veiculo_parado' | 'rota_desviada',
  entregadorId: string,
  mensagem: string,
  severidade: 'baixa' | 'media' | 'alta'
}
```

Dica para o Stream 3

Use `timer` combinado com `switchMap` e `Math.random()` para simular intervalos aleatórios. Se travar nessa parte, avance e volte depois — não deixe um stream te bloquear o resto do desafio.

Tarefa 2 — Transformações e Filtragens (25 pts)

Com os streams criados, aplique as seguintes transformações usando `pipe()` e operadores do RxJS:

1. **Filtrar velocidade suspeita:** A partir do `gps$`, crie um novo stream chamado `velocidadeSuspeita$` que emita apenas as leituras onde a velocidade for maior que 60 km/h.
2. **Contar por status:** A partir do `pedidos$`, crie um stream `statusCount$` que acumule e emita, a cada novo pedido, um objeto com a contagem total de cada status (ex: { entregue: 3, falhou: 1, em_rota: 5, coletado: 2 }). Use o operador `scan`.
3. **Filtrar alertas críticos:** A partir de `alertas$`, crie `alertasCriticos$` que emita apenas alertas com severidade 'alta' ou 'media'.
4. **Enriquecer dados de GPS:** A partir de `gps$`, use `map` para adicionar um campo `regiao` ao objeto, com base na latitude (`lat > 0 = 'Norte'`, `lat <= 0 = 'Sul'`). Nomeie esse stream como `gpsEnriquecido$`.

⚠ Atenção

Todos os streams desta tarefa devem ser criados a partir dos streams originais (`gps$`, `pedidos$`, `alertas$`), nunca recriados do zero. O objetivo é demonstrar composição reativa.

Tarefa 3 — Combinação de Streams (25 pts)

Aqui a complexidade aumenta. Você precisará combinar múltiplos streams para gerar visões consolidadas dos dados.

3.1 — Painel do Entregador

Objetivo: Crie um stream `painelEntregador$` que combine `gps$` e `pedidos$` e emita, sempre que qualquer um dos dois tiver uma atualização, um objeto consolidado com os dados mais recentes de um mesmo entregador.

```
// Saída esperada de painelEntregador$:
{
  entregadorId: string,
  ultimaLocalizacao: { lat, lng, velocidade },
  ultimoStatus: string,
  ultimaAtualizacao: Date
}
```

Considere que `gps$` e `pedidos$` emitem para entregadores diferentes. Você vai precisar pensar em como correlacionar os dados pelo `entregadorId`. Não existe uma única forma de fazer isso — pesquise e justifique sua escolha no README.

3.2 — Dashboard de Emergência

Crie um stream `emergencia$` que emita quando AMBAS as condições ocorrerem dentro de um intervalo de 5 segundos:

- Um alerta de severidade 'alta' for emitido
- O mesmo `entregadorId` tiver velocidade superior a 60 km/h no `gps$`

Esta é a parte mais difícil do desafio. Se travar, implemente parcialmente e documente o que tentou no README. Isso também conta pontos.

Tarefa 4 — Gerenciamento de Erros e Ciclo de Vida (20 pts)

4.1 — Simular e tratar erros

Modifique `pedidos$` para que, aleatoriamente (10% das vezes), ele lance um erro com a mensagem 'Falha na comunicação com o servidor'.

Em seguida, aplique tratamento de erro no pipeline para que:

- **O stream não quebre:** use `catchError` para capturar o erro e emitir um objeto de status especial `{ status: 'erro', mensagem: '...' }`.
- **Tentativas automáticas:** use `retry(3)` para tentar novamente até 3 vezes antes de acionar o `catchError`.

4.2 — Gerenciar o ciclo de vida

No arquivo `main.ts`, faça todas as subscrições de forma organizada:

- **Use um `Subject destroy$`** e o operador `takeUntil` para garantir que todos os streams são cancelados após 30 segundos de execução.
- **Ao finalizar**, exiba no console um resumo com o total de eventos processados por cada stream.

Tarefa 5 — Operador Customizado (10 pts)

Crie um operador customizado chamado `logComTimestamp` que possa ser usado dentro de qualquer `pipe()` e que:

- Exiba no console o valor emitido com o horário exato da emissão
- Deixe o valor passar sem alterá-lo (não deve interferir no stream)

- Aceite um parâmetro opcional `label: string` para identificar qual stream está sendo logado

Exemplo de uso esperado:

```
gps$.pipe(  
  logComTimestamp('GPS'),  
  filter(x => x.velocidade > 60)  
)  
.subscribe(...)  
  
// Saída esperada no console:  
// [GPS] 14:32:01.452 → { entregadorId: 'ENT-002', velocidade: 73, ... }
```

4. Critérios de Avaliação

Critério	Descrição	Pontos
Tarefas 1 a 5 implementadas	Funcionalidade conforme especificado	100 pts
Uso correto dos operadores	Escolha adequada e sem antipadrões	+10 pts
README explicativo	Justificativas das escolhas técnicas	+10 pts
Código limpo e organizado	Nomes descritivos, sem código morto	+5 pts
Tentativa documentada (T3.2)	Mesmo parcial, se bem explicada no README	+5 pts

5. Antipadrões que Causam Desconto

Os itens abaixo serão penalizados automaticamente na avaliação:

Antipadrão	Desconto
subscribe() dentro de subscribe()	-10 pts cada ocorrência
Não cancelar subscriptions (sem unsubscribe/takeUntil)	-10 pts
Recriar Observables ao invés de compor a partir dos originais	-5 pts
Usar Promises onde se pede Observables	-5 pts cada
Ignorar tratamento de erros	-10 pts

6. O que deve estar no README

O README é parte da entrega e será lido com atenção. Inclua:

5. Como rodar o projeto (comandos passo a passo)
6. Uma breve descrição do que cada stream faz
7. A justificativa para o operador escolhido na Tarefa 3.1 (por que combineLatest, withLatestFrom, ou outro?)
8. O que você tentou na Tarefa 3.2, mesmo que não tenha conseguido concluir
9. Uma dificuldade que você enfrentou e como resolveu (ou tentou resolver)

7. Dicas Finais

Recursos recomendados

rxjs.dev/api — Documentação oficial com exemplos interativos

rxmarbles.com — Visualize o comportamento dos operadores em diagramas marble

learnrxjs.io — Exemplos práticos com explicações simples

Estratégia sugerida

1. Comece pela Tarefa 1 e garanta que os três streams emitem dados no console.
2. Avance pelas Tarefas 2 e 4 — são mais diretas e valem metade da nota.
3. Enfrente a Tarefa 3 após ter confiança nos operadores básicos.
4. Deixe a Tarefa 5 (operador customizado) para o final.
5. Reserve tempo para o README — ele pode mudar sua nota.

Boa sorte — e lembre-se: travar faz parte do aprendizado.